Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., and Levy, O. LIMA: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36:55006–55021, 2023a. URL https://openreview.net/forum?id=KBMOKmX2he.

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023b. URL https://arxiv.org/abs/2311.07911.

A. Theoretical Properties of Task Complexity

The definition of Kolmogorov complexity is commonly attributed to Kolmogorov (1965), but was independently described by Solomonoff (1964) and Chaitin (1966), and is given in Equation (6). In this section, we list a number of properties of task complexity which are closely related to the notion of Kolmogorov complexity. First, we present three useful bounds on conditional and unconditional task complexity.

Remark A.1. *For any task $\mathtt{T}$ and performance δ, conditioning on a model θ can only reduce task complexity:*

$$\mathrm{C}(\mathtt{T}_\delta \mid \theta) \leq \mathrm{C}(\mathtt{T}_\delta) \tag{8}$$

Proof. Let $\mathsf{P}^\star$ be the shortest (input-free) program achieving $\mathtt{score}_\mathtt{T}(\mathsf{P}^\star) \geq \delta$, so $\mathrm{C}(\mathtt{T}_\delta) = \mathrm{len}(\mathsf{P}^\star)$. We can augment this program $\mathsf{P}^\star$ with an ignored input θ, and it would still solve task $\mathtt{T}$ at performance δ.[10] □

Remark A.2. *For any task $\mathtt{T}$, performance δ and model θ, (unconditional) task complexity is upper bounded by the task complexity conditioned on θ plus an overhead for encoding θ:*

$$\mathrm{C}(\mathtt{T}_\delta) \leq \mathrm{C}(\mathtt{T}_\delta \mid \theta) + \mathrm{K}(\theta) + O(1) \tag{9}$$

Proof. Let $\mathsf{P}^\star$ be the shortest program which, conditioned on θ, solves $\mathtt{T}_\delta$, i.e., $\mathtt{score}_\mathtt{T}(\mathsf{P}^\star_\theta) \geq \delta$. We thus have that $\mathrm{C}(\mathtt{T}_\delta \mid \mathsf{P}) = \mathrm{len}(\mathsf{P}^\star)$. We can construct a program P' that directly encodes θ (which requires $\mathrm{K}(\theta)$ bits), which it uses to execute $\mathsf{P}^\star$ (requiring $\mathrm{len}(\mathsf{P}^\star)$ bits plus an $O(1)$ overhead). Thus:

$$\mathrm{C}(\mathtt{T}_\delta) \leq \mathrm{len}(\mathsf{P}') = \mathrm{K}(\theta) + \mathrm{len}(\mathsf{P}^\star) + O(1) \tag{10a}$$

$$= \mathrm{K}(\theta) + \mathrm{C}(\mathtt{T}_\delta \mid \theta) + O(1) \tag{10b}$$

□

Remark A.3. *Task complexity is monotonic on the performance δ, meaning that, for any task $\mathtt{T}$ and performances $\delta_1 \leq \delta_2$:*

$$\mathrm{C}(\mathtt{T}_{\delta 1}) \leq \mathrm{C}(\mathtt{T}_{\delta 2}) \tag{11}$$

with the same holding for conditional task complexity.

Proof. Any program P that achieves performance δ_2 also achieves performance δ_1. Its length is therefore an upper bound on $\mathrm{C}(\mathtt{T}_{\delta 1})$. □

We now show that task complexity generalises Kolmogorov complexity, having it as a special case.

Remark A.4. *Task complexity generalises Kolmogorov complexity.*

Proof. Let s be any finite string. We write its Kolmogorov complexity as $\mathrm{C}(s)$. Now, let $\delta = 1$ and construct a task $\mathtt{T}$ with no inputs, and whose goal is solely to output this string s. Formally: $\mathcal{X} = \{\varepsilon\}$, $\mathcal{Y} = \{0, 1\}^*$, $p(\varepsilon) = 1$, and $\mathcal{S}(x, y)$ returns 1 if and only if $y = s$ and 0 otherwise. Then we have exactly: $\mathrm{C}(\mathtt{T}_\delta) = \mathrm{C}(s)$. □

As task complexity reduces to Kolmogorov complexity in a special case, it is easy to show it is both uncomputable and unbounded. Uncomputability means there does not exist a program such that, for all tasks $\mathtt{T}$ and all performances δ, it computes $\mathrm{C}(\mathtt{T}_\delta)$. Unboundedness means that there does not exist any constant k which is larger than all task complexities.

Corollary A.5. *Task complexity is uncomputable.*

Corollary A.6. *For any $k \in \mathbb{N}$, there exists a task $\mathtt{T}$ and performance δ such that $\mathrm{C}(\mathtt{T}_\delta) \geq k$.*

[10] Depending on the UTM formalisation, ignoring extra inputs may require an additional $O(1)$ term in the inequality.

B. How to compute len(P)

Our definitions in Section 3 rely on a Universal Turing Machine M and binary-string programs P; the size of program P is then simply its length in bits. In practice, our operationalisation into experiments (Section 5) relies on `python` being the machine M and on multiple files representing parts of the program and specific information (about the program's data or adapted parameters) which it loads. For both the data and inference-control methods, thus, our program P's length is thus the size in bits of the actually used Python script, plus the size of a separate file which stores its required data. For the parameter methods, our program P's length is the size in bits of the used Python script, plus the size of a separate file which stores the saved adapter parameters. The implementation of all of our measurements are in our repository.

The data file size. We encode any data used in our methods via arithmetic coding with the model θ, following Deletang et al. (2024). This is an efficient method to losslessly compress and decompress data if given access to an LLM. To speed up our experiments, we estimate the size of this encoded data as the negative log-likelihood of the forward pass of model θ on the data, as similarly done by prior work measuring the size in bits of data (Morris et al., 2025; Pezeshkpour, 2023).

The weights file size. We always train our models using 16 bit accuracy, and therefore we compute the size of the weights as 16 times the number of trainable parameters of the adapter. For Bayesian-LoRA, we compute the size of each adapter matrix as $q \times (r \times (d_{in} + d_{out}))$ where q is the learned quantisation level, r is the learned rank, and d_{in} and d_{out} are the dimensions for the matrix being adapted. Note that $r \times (d_{in} + d_{out})$ is the number of trained adapter parameters.

The Python script file size. Given some python file, e.g. a `main.py`, we can measure its storage size in bytes as `wc -c main.py`. However, this does not consider the size of the code to run any of its dependencies, e.g. `import transformers`. We will assume that any library (built-in or not) is part of the machine M, and therefore ignore its measurements. This rule could be abused, for example, by creating a library that contains all of the code for a program P of our experiments. We do not fall into this behaviour. In our experiments, the most niche and specific python library which we assume access to is gptzip, which implements the encoding of data via arithmetic coding with LLMs. The methods Base Model, ICL, URIAL, and Subset Training, LoRA and Bayesian-LoRA have, respectively, sizes 2952, 3704, 3704, 5704, 2832 and 8376 in bits.

C. Tasks

We are motivated to select tasks that: (i) cover a diverse set of NLP domains, and (ii) plausibly have high complexity $\mathrm{C}(\mathtt{T}_\delta)$ for some levels of performance. We select the following three tasks. We note that, to quantify the performance $\mathtt{score}_\mathtt{T}(\mathsf{P})$ in each task, we use sample averages over finite test sets as an approximation to the expectation over the task's distribution p.

Mathematical Reasoning. For this task, we use the GSM8K dataset (Cobbe et al., 2021). This dataset contains grade-school math word problems requiring multi-step arithmetic reasoning. We use MetaMath (Yu et al., 2024), which is an extended version of the training set, containing 395k examples. We measure the performance as accuracy of the final response.

Machine Translation. For this task, we use the English to French portion of the FLORES-200 dataset (Costa-Jussà et al., 2024). We draw the first 100k examples from the training set, and use BLEU (Papineni et al., 2002) as the performance metric.

Instruction Following. For this task, we use the IFEval dataset (Zhou et al., 2023b). The dataset contains queries as *'Explain concept X'* augmented with easily verifiable rules as *'use only lowercase'* or *'do not use commas'*. We measure performance as the average success of following these rules, and we verify each rule using the official implementation of the dataset. We generate a training set of 37k examples, using the Tulu 3 data (Lambert et al., 2025), which we detail in the next section.

C.1. Generating Instruction Following Data

The original work about the superficiality of LLM adaptation focused on adapting them to follow instructions, and therefore, we found it important to add this to our set pf analysed tasks. Evaluating instruction following behaviour is not necessarily an easy task, and previous work relied on the use of expensive human annotations or LLM judges (Zhou et al., 2023a; Hewitt et al., 2024; Lin et al., 2024). Rather, we opted for using a simpler and more reliable evaluation method: IFEval (Zhou et al., 2023b), which implements 'verifiable instructions'. However, IFEval does not provide a training set, which is the